© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_11

11. Agents Assemble! Building Autonomous Workflows

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to understand agents and their applications using Spring AI.

Introduction

In recent years, the integration of large language models (LLMs) and agents into software systems has revolutionized the way people interact with AI. While LLMs have demonstrated powerful capabilities in natural language understanding and generation, their true potential is unlocked when combined with agent-based systems. These agents, powered by LLMs, can autonomously navigate workflows, make decisions, and interact with a variety of tools and systems to achieve specific goals.

Spring AI offers a robust framework for building such agentic systems, allowing developers to construct, deploy, and scale AI-driven agents in a seamless and maintainable manner. This chapter focuses on understanding agents and their applications within the Spring AI ecosystem, exploring fundamental patterns such as chain workflows, parallelization, routing, and more. Through the use of Spring AI's tools like the ChatClient API and the Advisors API, developers can build highly flexible, modular, and effective agents.

In this chapter, you will dive into key components like agentic workflows, agentic workers, and the Advisors API to see how these elements contribute to building efficient and autonomous AI agents. Whether you're new to AI-driven agents or looking to enhance your existing knowledge, this chapter will provide a clear understanding of how to utilize Spring AI's capabilities to build and manage effective agents.

Agentic AI

An *agent* in the context of artificial intelligence (AI) is a software component or system that acts autonomously to perform tasks, make decisions, or solve problems based on the environment or situation it encounters. Agents can perceive their environment, reason about it, and take actions to achieve a specific goal. They are typically built to operate with a certain level of autonomy, meaning they don't require constant human intervention to carry out their tasks. Figure 11-1 shows the key components and interactions.

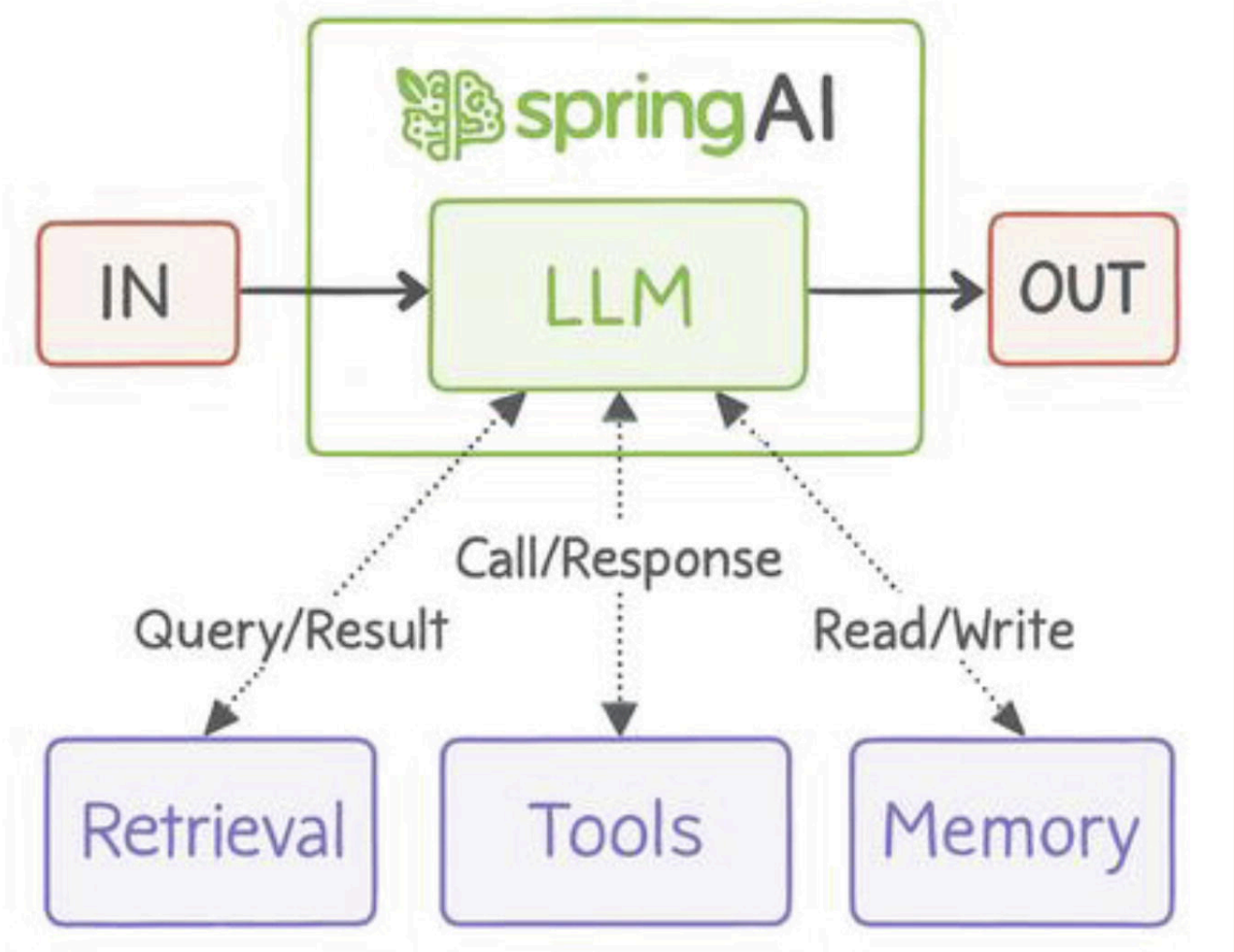


Figure 11-1 Agentic AI ecosystem with components and interaction

Key Characteristics of an Agent

The following are the key characteristics:

- **Autonomy**: An agent operates independently to
- perform tasks and make decisions without constant human input.

- **Perception**: An agent has sensors or mechanisms to perceive its environment, gathering information about the current state.
- **Action**: Based on its perception, the agent can take actions (e.g., executing commands, making decisions, or interacting with other systems) to influence the environment or achieve a goal.
- **Reasoning**: Agents often have some level of reasoning capability, allowing them to interpret data, make informed decisions, and adjust actions as needed.
- **Goal-Oriented**: Agents usually operate with a specific goal or set of goals in mind, whether it's answering a query, solving a problem, or completing a task.

Types of Agents

There are several different types of agents:

- **Simple agents**: These are basic agents that perform tasks by following a set of predefined rules. They often work in environments where conditions are predictable and controlled.
- **Autonomous agents**: These agents have the ability to make decisions on their own and adapt to dynamic, uncertain, or complex environments.
- **Intelligent agents**: These agents are capable of more advanced behaviors, such as learning, reasoning, and problem-solving. They can improve their performance over time based on experience or external feedback (e.g., machine learning).

Applications of Agents

These are ways agents are used:

- **Virtual assistants**: Such as Siri, Alexa, and Google Assistant, which autonomously process requests, learn user preferences, and act based on verbal commands
- **Robots**: Autonomous robots like drones or vacuum cleaners that can navigate and complete tasks without human intervention
- **Recommendation systems**: Agents that analyze user behavior and recommend products or services, such as those on Amazon or Netflix
- **AI chatbots**: AI agents designed to interact with users in a conversational manner to answer questions or resolve issues, often used in customer service

In modern software development, agents are often used in AI applications where decision-making, automation, and interaction with dynamic environments are required. They are particularly powerful when combined with technologies like LLMs and sophisticated workflows, as seen in frameworks like Spring AI.

The fundamental idea behind agents is to use a language model to identify a sequence of actions. In contrast to chains, where actions are predefined, agents rely on the language model as a reasoning engine to decide both the actions (tools) to take and the order in which to perform them.

Defining Concepts

Agent: An agent serves as the primary decision-maker, responsible for determining the next steps. It is driven by a language model and a prompt. The inputs include:

- **Tools**: Descriptions of the available actions
- **User Input**: The overall objective or task
- **Intermediate Steps**: A series of (action, tool output) pairs executed in sequence to achieve the user's goal

The output produced by the agent specifies the next action(s) to be taken or provides the final response for the user. Each action details the tool (method name) to be used and its relevant input parameters.

Tools: Tools are the methods that an agent can invoke. There are two key factors when utilizing tools:

- Providing the agent with access to the appropriate tools
- Describing these tools in a way that maximizes their usefulness to the agent

Neglecting either of these elements can impair the agent's performance. If the agent does not have access to the correct tools, it won't be able to fulfill its objectives, and if the tools are poorly described, the agent will struggle to use them effectively.

NoteWhen building an agent, it is important to set the model temperature to 0, which ensures that the agent consistently chooses the most probable action. Additionally, the descriptions of the tools available to the agent should be clear and detailed to maximize their effectiveness. Finally, it is essential to organize the steps in the prompt in the desired execution order, ensuring that the agent follows a logical and efficient sequence when performing its tasks.

Components of Agentic Systems

Agents are systems that intelligently perform tasks, ranging from executing simple workflows to pursuing more complex, open-ended objectives. Building an effective agent involves assembling components from multiple domains, such as models, tools,

knowledge and memory, audio and speech, guardrails, and orchestration. These components work together to create a robust agentic platform, each contributing to the overall functionality and adaptability of the agent.

Models

At the heart of any agent is its model, which serves as the core intelligence, enabling the agent to reason, make decisions, and process various types of information. Some models are specifically designed for long-term planning and reasoning, making them ideal for tackling difficult tasks that require strategic thought and deep problem-solving. On the other hand, certain models are optimized for agentic execution, focusing on real-time task completion with high efficiency. There are also models that strike a balance between agentic capabilities and latency, making them well-suited for tasks that require both flexibility and speed. For applications that demand minimal delays, low-latency models are essential, ensuring the agent can interact and respond quickly, especially in real-time scenarios. These diverse models work together to provide a range of capabilities that enable agents to engage with the world, support multimodal inputs such as text, images, and audio, and adapt to different contexts and tasks.

Tools

Tools are essential for enabling agents to interact with their environment and execute the tasks they are designed for. They provide the interface through which agents access external systems or carry out operations beyond their internal model. Function calling allows agents to interact with code defined by developers, enabling the execution of specific functions that are critical to the agent's operation. For agents that need to gather up-to-date information, tools like web search enable them to fetch current data from the Internet. Similarly, agents can use file search tools to perform semantic searches across large datasets or documents, making it easier to retrieve relevant information. Additionally, tools that allow agents to understand and control a computer or browser empower them to interact with and manipulate other systems, offering a broad spectrum of capabilities. By integrating these tools, agents become more versatile and capable of handling a wide variety of tasks, from simple queries to complex system manipulations.

Knowledge and Memory

For an agent to be effective over time, it needs to access external knowledge and retain information from past interactions. Knowledge and memory components are essential for augmenting the agent's capabilities beyond its initial training data. Vector stores are

one key component that allows agents to search through documents semantically, enabling them to retrieve contextually relevant information dynamically during interactions. This capability is essential for tasks that require up-to-date or highly specific data. In addition to vector stores, embeddings represent data in a way that makes it efficient to retrieve, even for large datasets, ensuring that agents can access relevant information quickly. These components not only improve the agent's ability to perform tasks at runtime but also support long-term memory, allowing agents to adapt to new knowledge, retain information across sessions, and respond appropriately based on historical interactions.

Guardrails

Guardrails are critical for ensuring that agents behave safely, ethically, and consistently, particularly in real-world applications where their behavior may impact users or systems. These mechanisms are designed to prevent agents from engaging in unsafe, irrelevant, or harmful actions, which is especially important when agents are deployed in production environments. Moderation tools automatically filter out harmful or inappropriate content, helping to maintain a positive and safe user experience. Additionally, instruction hierarchies give developers the ability to control the behavior of agents by prioritizing specific prompts or guidelines. This ensures that the agent follows desired patterns of behavior and avoids unwanted actions, even in complex or ambiguous situations. By setting clear boundaries through guardrails, developers can maintain control over the agent's operations, ensuring it acts within the intended ethical and functional guidelines.

Orchestration

Building and managing agents is an iterative process that involves developing, deploying, and continuously improving the system. Orchestration tools are essential for monitoring and refining the performance of agents over time. The Agent SDK provides developers with the necessary tools to create, test, and deploy agents, ensuring they are built effectively from the start. Once deployed, agents can be monitored using tracing and evaluation tools, which allow developers to track their performance and identify areas for improvement. Fine-tuning is another crucial aspect of orchestration, enabling agents to be adapted over time to perform specific tasks or adjust to new challenges. Through effective orchestration, agents can be maintained, updated, and optimized to ensure they continue to meet the evolving needs of users and organizations.

By combining these components—models, tools, knowledge, guardrails, and orchestration—agents can be built and refined to handle a wide range of tasks in dynamic, real-

world environments. Each component plays a vital role in enabling agents to perform intelligently, safely, and effectively, ensuring their success in a variety of applications.

Agentic Workflows and Patterns

In current practice, agent-based AI systems are widely used to automate tasks, make decisions, and solve problems across various domains. Frameworks like LangChain4j are commonly used for building these systems, but they do not currently support high-level abstractions like “agents” found in AutoGen or CrewAI, which are specifically designed for building multi-agent systems. This limitation means that within Java frameworks like LangChain4j, constructing complex multi-agent systems is not as straightforward. However, it is still possible to implement similar patterns manually. In the upcoming sections, we will explore how these patterns can be constructed, providing a foundation for building more sophisticated agent systems even within the current framework constraints.

In the context of building agent-based AI systems, there are three primary architectural patterns that can be utilized: Workflows, Agents, and a combination of both, known as Hybrid Systems. Understanding the distinction between these patterns helps in deciding which approach is best suited for a given task.

1. **Workflows**: In this pattern, systems are designed where LLMs and tools are orchestrated through predefined, static code paths. These systems follow a prescriptive model, where the sequence of actions, tools, and processes are explicitly defined by the developer. This approach is ideal for tasks that require predictability, consistency, and strict control over the execution process. By using workflows, developers ensure that the agent follows a set sequence of operations, making it suitable for well-understood tasks like data retrieval, file searching, and system operations that don't require decision-making flexibility.
2. **Agents**: Unlike workflows, agents are dynamic systems where LLMs autonomously direct their own processes and choose the tools they need based on the context or input. This approach offers more flexibility, allowing the agent to make decisions about what actions to take next, making it suitable for complex, open-ended tasks. Agents are ideal for scenarios that require adaptability, such as when the agent needs to handle unknown or variable inputs and adjust its course of action accordingly. The agent system excels in situations where tasks are not strictly defined in advance and require ongoing reasoning and decision-making.
3. **Hybrid Systems**: The hybrid approach combines elements of both workflows and agents. In a hybrid system, predefined workflows guide the agent through specific tasks, but the agent still retains the ability to make decisions and dynamically

choose between different workflows or tools based on the context. This combination offers the best of both worlds: the structured predictability of workflows for well-defined tasks and the flexibility of agents for tasks that require adaptability and decision-making. Hybrid systems are useful in situations where certain parts of a task can be well-defined and controlled through workflows, but other parts may require the agent to autonomously decide on the best course of action based on the available information.

While you've already explored the use of tools within agent systems—where LLMs autonomously decide to fetch the date, set alarms, or call APIs for weather updates—the focus in this section will be on *workflow patterns*. You will dive deeper into how workflows can be effectively structured and managed, especially in scenarios where predictability, repeatability, and structured execution are critical. I'll also briefly touch on how hybrid systems can leverage the strengths of both workflows and agents to address complex use cases.

The primary takeaway is that while the idea of fully autonomous agents is certainly attractive, workflows often offer superior predictability and consistency, especially for tasks that are well-defined. This is particularly important in enterprise environments, where factors like reliability, control, and ease of maintenance are essential for successful deployment. Workflows allow organizations to implement structured, repeatable processes that minimize risks and ensure a high level of operational stability.

In contrast, fully autonomous agents, while offering more flexibility and decision-making capabilities, can introduce unpredictability that may not be ideal in contexts where clear, reliable outcomes are necessary. This makes workflows a better choice for many enterprise scenarios, where a controlled approach is paramount.

To illustrate how these concepts can be applied in real-world systems, let's explore how Spring AI implements these patterns through five fundamental workflow structures. Each of these patterns is designed to address specific use cases and optimize the execution of tasks in various business applications. By understanding how these patterns are applied, you can better appreciate the balance between flexibility and structure that makes workflows such an important tool in building reliable, maintainable systems.

Chain Workflow

The Chain Workflow pattern is a powerful method for breaking down complex tasks into simpler, more manageable steps (see Figure 11-2).

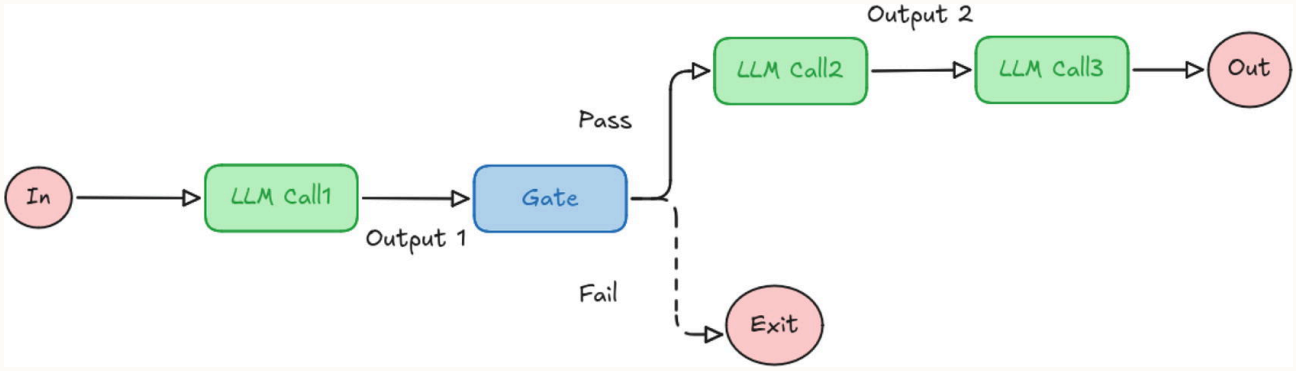


Figure 11-2 Chain Workflow pattern

In this pattern, each step in the workflow relies on the output of the previous step, creating a sequence of actions that build upon each other. This approach is ideal for tasks where the process needs to follow a clear, linear sequence, allowing for more accurate and controlled execution.

When to Use Chain Workflow

The following are times to use Chain Workflow:

- **Tasks with clear sequential steps**: When the task at hand involves multiple stages that need to be processed in a specific order.
- **Trading latency for accuracy**: Sometimes, breaking down a task into smaller steps improves the overall accuracy of the result, even if it introduces some latency.
- **Stepwise dependency**: Each step's output is used as input for the subsequent step, ensuring that the process evolves logically.

Let's break down the code and explain how it implements the Chain Workflow pattern.

File: src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import com.example.demo.workflow.*;
import com.fasterxml.jackson.core.JsonProcessingException;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import java.util.List;
import java.util.Map;
@SpringBootApplication
public class DemoApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
    // Bean to initialize and run the chat process as a command line runner
    @Bean
    public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatMo
del) {
        return args -> {
            // Creating the chat client with the provided chat model
            ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
            // Call the 'chain' method to execute the workflow
            String response = chain(
                    "What is 2+2?",  // The user input to the chain workflow
                    new String[]{    // Predefined system prompts to guide the workflow
                            "Think through it.",
                            "Validate if the result is scientifically presented.",
                            "Share only the output."
                    },
                    chatClient);   // Pass in the chat client to interact with the AI model
            // Print the final result
            System.out.println("Response: " + response);
        };
    }
    // Method that implements the Chain Workflow
    private String chain(String userInput, String[] systemPrompts, ChatClient chatClient)
{
        String response = userInput;  // Start with the user input as the initial response
        // Iterate through the system prompts, applying them sequentially
        for (String prompt : systemPrompts) {
            // Concatenate the current prompt with the previous response (or initial input)
            String input = String.format("%s\n %s", prompt, response);
            // Pass the concatenated input to the chat client and get the updated response
            response = chatClient.prompt(input).call().content();  // Make the call to the LLM
        }
        return response;  // Return the final output after all steps
    }
}
```

The code demonstrates the implementation of a Chain Workflow pattern, where a sequence of steps is executed in a predefined order, with each step building upon the output of the previous one. The workflow begins by defining a CommandLineRunner bean in a Spring Boot application. This bean is responsible for initiating the workflow when the application starts. Within this bean, a ChatClient is created using the OpenAiChatModel, which serves as the interface to interact with the LLM.

The core of the workflow lies in the chain method, which takes the user input, a set of system prompts, and the chat client as parameters. The method starts by initializing the response with the user's input (in this case, “What is 2+2?”). Then, a loop iterates through each of the system prompts provided in the systemPrompts array. During each iteration, the current prompt is concatenated with the previous response to form the input for the LLM. The chatClient.prompt(input).call().content() call sends this input to the LLM and receives the updated response, which is then passed to the next iteration.

Each prompt in the array guides the agent through a specific step, such as “Think through it,” “Validate if the result is scientifically presented,” and “Share only the output.” These steps are designed to refine the answer progressively. Once all the prompts are processed, the method returns the final response, which represents the output of the entire workflow after all steps have been executed. The final result, in this case, is the output of the mathematical calculation, ensuring it was properly reasoned and validated.

This approach effectively demonstrates how the Chain Workflow pattern can be used to break down a task into manageable steps, where each step is dependent on the outcome of the previous one. It ensures the task is performed systematically, with clear responsibilities assigned to each stage, resulting in a more accurate and controlled output. The chain is also highly extensible, as additional steps can be easily added to the workflow as needed.

What’s Happening in the Workflow?

This is what happens:

- The Chain Workflow begins with a user input and proceeds through a series of system prompts. Each prompt builds on the previous response, ensuring that the task is completed in well-defined steps.
- The agent is guided through the process of reasoning, validation, and output generation, with each stage relying on the result from the previous one. This approach ensures a more accurate and controlled final result.
- The response from the LLM is updated at each step, demonstrating the chain’s ability to modify the task dynamically while maintaining control over the process.

This implementation highlights several key principles of the Chain Workflow:

- **Focused responsibilities**: Each step has a clear responsibility (e.g., thinking through the problem, validating the result, sharing only the output).
- **Sequential dependency**: The output from one step becomes the input for the next, creating a clear flow of logic.

- **Extensibility**: The chain can easily be extended to include additional steps or modified to handle more complex tasks.

This pattern is ideal for structured tasks where each stage builds on the previous one, ensuring that the final result is both accurate and comprehensive.

Parallelization Workflow

The Parallelization Workflow pattern leverages concurrent processing to handle multiple tasks simultaneously, improving efficiency and reducing processing time (see Figure 11-3).

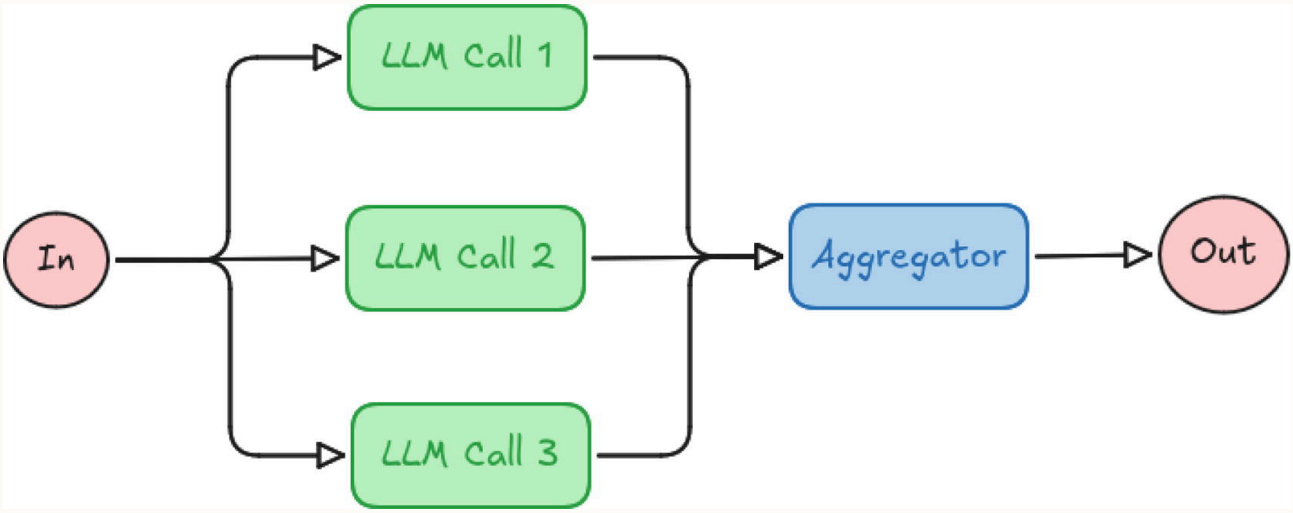


Figure 11-3 Parallelization Workflow

This pattern is particularly useful in scenarios where large volumes of similar but independent tasks need to be processed or when tasks require multiple perspectives. There are two main variations of parallelization:

- **Sectioning**: In this variation, a task is broken down into smaller, independent subtasks that can be processed in parallel. This allows the system to handle each subtask concurrently, speeding up the overall processing time.
- **Voting**: In this variation, multiple instances of the same task are run simultaneously, and the outputs are aggregated to form a final result based on consensus.

This workflow is ideal when processing time is critical, and tasks can be parallelized. It allows for simultaneous execution of multiple LLM calls, with the results being aggregated afterward to provide the final output. For example, in stakeholder analysis, the system can analyze different stakeholder groups concurrently and then combine the results for an overall assessment.

The following code demonstrates the Parallelization Workflow in action. The commandLineRunner method is executed when the application starts, initiating a parallel

processing task using a ChatClient.

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatMode
l) {
    return args -> {
        System.out.println("Parallelization Workflow:");
        List<String> parallelResponse = parallel(chatClient);
        parallelResponse.forEach(System.out::println); // Output the responses for each sta
keholder group
    };
}
private List<String> parallel(ChatClient chatClient) {
    return new ParallelizationWorkflow(chatClient)
            .parallel(
                    "Analyze how market changes will impact %s stakeholder group.",  // Prom
pt to analyze market changes
                    List.of(
                            "Customers",   // Stakeholder 1
                            "Employees",   // Stakeholder 2
                            "Investors",   // Stakeholder 3
                            "Suppliers"    // Stakeholder 4
                    )
            );
}
```

In this code, the parallel method is called, which initializes the ParallelizationWorkflow and runs the analysis for each stakeholder group in parallel. The List.of() method is used to define the stakeholders, and each group is passed to the LLM for concurrent processing.

```
package com.example.demo.workflow;
import org.springframework.ai.chat.client.ChatClient;
import java.util.List;
public class ParallelizationWorkflow {
    private final ChatClient chatClient;
   // Constructor to initialize ChatClient
    public ParallelizationWorkflow(ChatClient chatClient) {
        this.chatClient = chatClient;
    }
   // Method to process tasks in parallel
    public List<String> parallel(String prompt, List<String> stakeholders) {
        return stakeholders.parallelStream()  // Use parallelStream to process items concurr
ently
```

```
            .map(stakeholder -> this.chatClient.prompt(String.format(prompt, stakehold
er)).call().content()) // Map each stakeholder to a LLM call
            .toList();  // Collect the results into a List
    }
}
```

The ParallelizationWorkflow class utilizes Java's parallelStream() to process each stakeholder group concurrently. For each stakeholder, the chatClient.prompt() method sends a request to the LLM with the respective stakeholder's name. The results are then gathered into a list and returned.

What's Happening in the Process?

This is what happened:

1. **CommandLineRunner**: This is used to execute the parallelization logic when the application starts. It runs the parallel() method and prints the resulting analysis for each stakeholder group.
2. **Parallel Execution**: The parallelStream() method is used to process the list of stakeholders in parallel. Each stakeholder's group is analyzed independently, allowing the system to handle multiple tasks concurrently. This significantly speeds up the process, especially when dealing with a large number of similar tasks.
3. **LLM Call**: Each iteration of the parallelStream() sends a prompt to the LLM for analysis. The prompt asks the LLM to analyze how market changes will impact each stakeholder group.
4. **Aggregation**: After all parallel tasks are completed, the results from each LLM call are collected into a list, providing a comprehensive analysis of how market changes affect each stakeholder group.

The Parallelization Workflow pattern is particularly beneficial for tasks that can be broken down into independent units, such as analyzing multiple groups or processing large datasets, as it optimizes performance by executing tasks concurrently rather than sequentially.

Routing Workflow

The Routing Workflow pattern is designed to implement intelligent task distribution, enabling specialized handling for different types of input (see Figure 11-4).

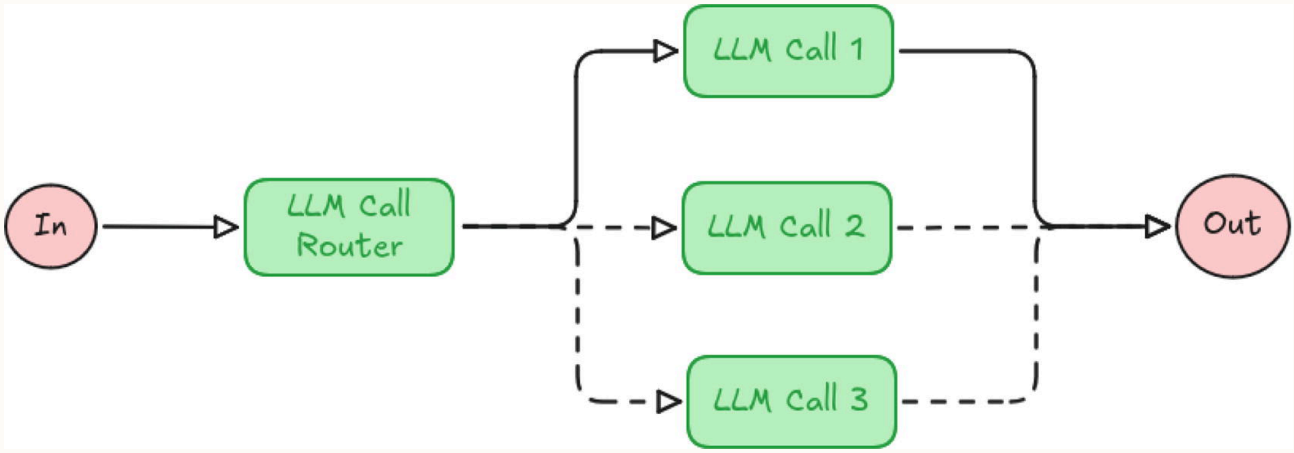


Figure 11-4 Routing Workflow

This pattern is particularly useful for complex tasks where various inputs need to be processed by specialized systems or processes. The core idea is to analyze the input content using an LLM and route it to the appropriate specialized prompt or handler for processing.

When to Use

This is when to use it:

- **Complex tasks with distinct categories of input**: When the task involves different categories of input, each requiring specific handling, such as customer inquiries, technical issues, or billing problems.
- **Specialized processing for different inputs**: When certain types of input require unique processing steps or specialized knowledge, this pattern ensures that the right expertise is applied.
- **Accurate classification**: If the system can accurately classify input into distinct categories, the routing pattern ensures that the task is sent to the right handler.

In the Routing Workflow pattern, an agent first analyzes the input and then determines the most appropriate route by matching it to a predefined set of categories or routes. This decision is made dynamically based on the content of the input, and the agent then proceeds with the task using the most relevant process.

Here's a breakdown of how the Routing Workflow pattern is implemented in the code:

```
package com.example.demo.workflow;
import org.springframework.ai.chat.client.ChatClient;
import java.util.Map;
import java.util.Objects;
public class RoutingWorkflow {
```

```
    public RoutingWorkflow(ChatClient chatClient) {
    }
    // The 'route' method determines which task handler should be invoked based on the inp
ut
    public String route(String input, Map<String, String> routes, ChatClient chatClient) {
        // Get all keys from the routes map (these represent different task types or categorie
s)
        StringBuilder keys = new StringBuilder();
        for (String key : routes.keySet()) {
            keys.append(key).append(", ");
        }
        // Remove the last comma and space
        if (!keys.isEmpty()) {
            keys.setLength(keys.length() - 2);
        }
        // Ask the LLM to determine the appropriate route based on the input
        String routeKey = Objects.requireNonNull(chatClient.prompt("Determine the rout
e for the input: " + input + ". Available routes: " + keys.toString() + ". Just respond wit
h the route name.")
                        .call()
                        .content())
                .trim()
                .toLowerCase();  // Get the route name from the LLM response
        String routeMessage = routes.get(routeKey);  // Retrieve the corresponding handler
for the selected route
        return chatClient.prompt(routeMessage).call().content();  // Execute the selected r
oute's handler and return the result
    }
}
```

What's Happening in the Process?

This is what happens:

1. **RoutingWorkflow class**: This class contains the logic for the routing workflow. It takes the user input and maps it to one of several predefined routes. The route method dynamically determines which route to use based on the input.
2. **Route determination**: The method first constructs a string of available routes by iterating over the routes map. This map contains keys that represent different categories of tasks (e.g., "billing", "technical", "general"). These categories will guide the LLM in determining the appropriate route.
3. **LLM decision**: The chatClient.prompt() sends a request to the LLM to analyze the input and decide which route it belongs to. The LLM is prompted with the input along

with a list of available routes. The LLM's response is expected to be the route name, which corresponds to one of the keys in the routes map.

4. **Executing the selected route**: Once the route is determined, the code retrieves the corresponding message from the routes map and sends it to the LLM for processing. The result from the LLM's response is returned as the output of the workflow.

Here's how the RoutingWorkflow is used in practice:

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatMode
l) {
   return args -> {
      ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
      String response = chain(
            "What is 2+2?",
            new String[]{
                  "Think through it.",
                  "Validate if the results is scientifically presented.",
                  "Share only the output."
            },
            chatClient);
     System.out.println("Response: " + response);
      System.out.println("Parallelization Workflow:");
     List<String> parallelResponse = parallel(chatClient);
      parallelResponse.forEach(System.out::println);
     System.out.println("Routing Workflow:");
      String routeResponse = route(chatClient);
      System.out.println("Response: " + routeResponse);
   }
   private String route(ChatClient chatClient) {
      RoutingWorkflow workflow = new RoutingWorkflow(chatClient);
      Map<String, String> routes = Map.of(
            "billing", "You are a billing specialist. Help resolve billing issues...",
            "technical", "You are a technical support engineer. Help solve technical probl
ems...",
            "general", "You are a customer service representative. Help with general inqu
iries..."
      );
      String input = "My account was charged twice last week";  // Input representing a b
illing issue
      return workflow.route(input, routes, chatClient);  // Call the routing workflow with
the input and routes
   }
```

Key Insights:

These are the key insights:

- **Task routing**: The routing workflow allows different types of tasks (billing, technical support, general inquiries) to be handled by specialized processes. By routing the input to the appropriate handler, the system can ensure that each task is dealt with by the right expertise.
- **Dynamic classification**: The LLM dynamically classifies the input and determines the appropriate route, making the system highly adaptable to varying input types.
- **Flexible and scalable**: This approach is flexible, allowing for easy addition of new routes and specialized handlers. The system can scale to accommodate new categories of tasks as needed.

This routing pattern is especially useful in customer support or other domains where different types of requests require different types of processing or expertise. It ensures that each request is processed efficiently by the most appropriate handler, improving the overall effectiveness of the agent system.

Agentic Workers

Agentic workers are AI systems capable of autonomously planning, deciding, and acting to achieve goals with minimal human oversight. They can break down complex tasks, reason through each step, and adapt as conditions change. Unlike traditional automation, they show initiative by identifying problems, proposing solutions, and coordinating workflows. By managing repetitive or cognitively heavy tasks, agentic workers free humans to focus on creativity, strategy, and higher-value work.

Orchestrator-Workers

The Orchestrator-Workers pattern is designed to implement more complex agent-like behaviors while maintaining control over the task decomposition and execution process (see Figure 11-5).

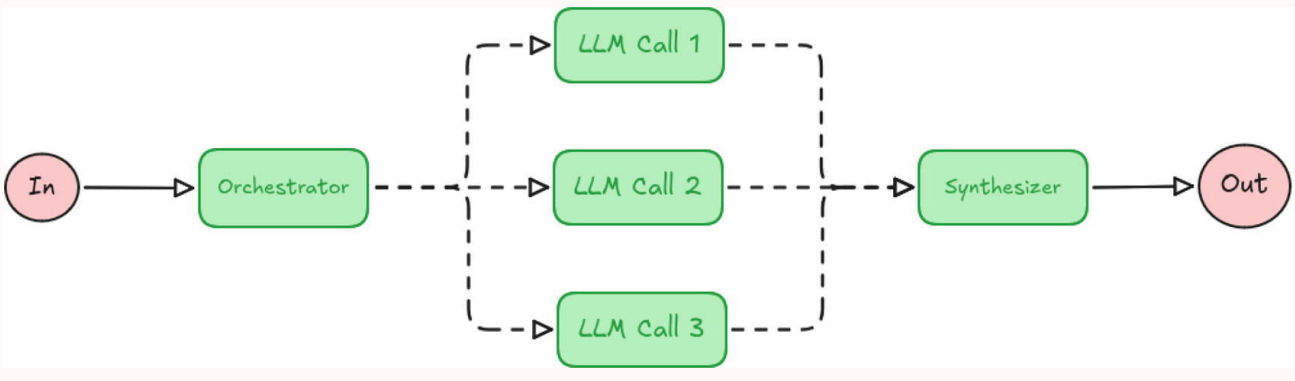


Figure 11-5 Orchestrator workers

This pattern involves a central orchestrator that divides a task into specialized subtasks, which are then handled by different workers. The workers, each specialized in a specific aspect of the task, execute their respective tasks concurrently, and their outputs are combined by the orchestrator to produce the final result.

This pattern is ideal for scenarios where tasks can't be easily predicted up front or when different perspectives or approaches are needed to address various aspects of a complex problem. The orchestrator ensures that the right worker handles the appropriate part of the task, and it can also manage situations where multiple workers need to collaborate to achieve the final outcome.

When to Use

This is when to use this pattern:

- **Complex tasks with unpredictable subtasks**: When the task at hand involves multiple facets or requires a combination of specialized expertise to complete.
- **Tasks requiring different approaches or perspectives**: When different methods or perspectives are needed to solve a problem, such as creating both technical and user-friendly documentation for an API.
- **Adaptive problem-solving**: When the task may evolve or require real-time adjustments based on the results of ongoing work.

The following code demonstrates how to implement the Orchestrator-Workers pattern using Spring AI's ChatClient for LLM interactions. The orchestrator divides the task of generating both technical and user-friendly documentation for a REST API endpoint into two separate tasks, handled by specialized workers.

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatMode
l) {
    return args -> {
        ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
        System.out.println("Orchestrator Workers Workflow:");
        orchestrate(chatClient);  // Execute the orchestrator workflow
    };
}
private void orchestrate(ChatClient chatClient) throws JsonProcessingException {
    OrchestratorWorkersWorkflow workflow = new OrchestratorWorkersWorkflow(chat
Client);
```

```
    WorkerResponse response = workflow.process(
            "Generate both technical and user-friendly documentation for a REST API endp
oint"
    );
    // Output the analysis and worker responses
    System.out.println("Analysis: " + response.analysis());
    System.out.println("Worker Outputs: " + response.workerResponses());
}
```

OrchestratorWorkersWorkflow Class

Here is what the OrchestratorWorkersWorkflow class looks like:

```
package com.example.demo.workflow;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.ai.chat.client.ChatClient;
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;
public class OrchestratorWorkersWorkflow {
    private final ChatClient chatClient;
    private final WorkerResponse workerResponse;
    public OrchestratorWorkersWorkflow(ChatClient chatClient) {
        this.chatClient = chatClient;
        this.workerResponse = new WorkerResponse();
    }
    public WorkerResponse process(String s) throws JsonProcessingException {
        // Orchestrator determines which workers to use based on the input
        String response = Objects.requireNonNull(this.chatClient.prompt("""
                There are two types of documentation for a REST API endpoint: technic
al and user-friendly.
                Technical documentation is intended for developers and includes detail
s about the API's functionality, parameters, and response formats.
                User-friendly documentation is intended for end-users and includes inf
ormation about how to use the API, examples, and best practices.
                There are two types of workers: technical and user-friendly. Either one
or Both the workers can be used simultaneously to generate documentation.
                The technical worker generates technical documentation, while the use
r-friendly worker generates user-friendly documentation.
                The user will provide a prompt that specifies the type of documentatio
n they want.
                Generate only a JSON string that contains the following fields without
any formatting or extra text:
```

```
                - "technical": a boolean value that indicates whether the technical doc
umentation should be generated
                - "userFriendly": a boolean value that indicates whether the user-frien
dly documentation should be generated
                The user will provide a prompt that specifies the type of documentatio
n they want:
                """)
            .user(s)
            .call()
            .content())
        .replaceAll("^```json\n|\n```$", "");
    // Parse the JSON response to determine whether to use each worker
    Map<String, Boolean> responseMap = new ObjectMapper().readValue(response, n
ew TypeReference<Map<String, Boolean>>() {});
    // Run the workers concurrently
    CompletableFuture<Void> technicalFuture = CompletableFuture.runAsync(() -> {
      if (responseMap.getOrDefault("technical", false)) {
        technicalWorker(chatClient);  // Generate technical documentation if required
      }
    });
    CompletableFuture<Void> userFriendlyFuture = CompletableFuture.runAsync(() -
> {
      if (responseMap.getOrDefault("userFriendly", false)) {
        userFriendlyWorker(chatClient);  // Generate user-friendly documentation if req
uired
      }
    });
    // Wait for both worker tasks to finish
    CompletableFuture.allOf(technicalFuture, userFriendlyFuture).join();
    // Orchestrator analyzes the results and returns a summary
    String analysis = this.chatClient.prompt("Analyze the response and provide a sum
mary of the documentation generated")
        .user(s)
        .call()
        .content();
    workerResponse.setAnalysis(analysis);
    return workerResponse;  // Return the orchestrated result
  }
  // Technical worker to generate technical documentation
  public String technicalWorker(ChatClient chatClient) {
    String response = this.chatClient.prompt("Generate technical documentation for a
REST API endpoint")
        .call()
        .content();
    workerResponse.setWorkerResponses(response);
    return response;
```

```
    }
    // User-friendly worker to generate user-friendly documentation
    public String userFriendlyWorker(ChatClient chatClient) {
        String response = this.chatClient.prompt("Generate user-friendly documentation f
or a REST API endpoint")
                .call()
                .content();
        workerResponse.setWorkerResponses(response);
        return response;
    }
}
```

What's Happening in the Process?

This is what is happening:

1. **CommandLineRunner**: The commandLineRunner method initiates the orchestration process when the application starts. It calls the orchestrate() method, which in turn invokes the orchestrator workflow.
2. **Orchestrator workflow**: The process() method in the OrchestratorWorkersWorkflow class orchestrates the generation of both technical and user-friendly documentation. It starts by using an LLM to analyze the user's request and determine which type of documentation (technical or user-friendly) is needed. This decision is made based on a JSON response that specifies whether to generate each type of documentation.
3. **Concurrent worker execution**: The technical and user-friendly documentation tasks are executed concurrently using CompletableFuture.runAsync(). This allows the system to generate both types of documentation simultaneously, improving efficiency.
4. **Worker response collection**: After the workers finish their tasks, the orchestrator synthesizes their outputs. The technicalWorker() and userFriendlyWorker() methods are responsible for generating the respective documentation types. Once both workers complete their tasks, the orchestrator analyzes the results and provides a summary.
5. **Final output**: The WorkerResponse object contains the results from both the workers, including an analysis of the generated documentation. The orchestrator ensures that all tasks are completed before returning the final output.

Key Insights

These are key insights:

- **Task decomposition**: The orchestrator divides the task into multiple subtasks, each handled by a specialized worker (technical and user-friendly).
- **Concurrency**: The workers are executed concurrently, improving the efficiency of the system when multiple tasks need to be performed simultaneously.
- **Synthesis**: The orchestrator combines the results of the workers and provides an overall analysis of the generated documentation.
- **Scalability**: This approach can be extended to include additional workers for other types of tasks, making the system adaptable to a wide range of use cases.

The Orchestrator-Workers pattern is ideal for scenarios where complex tasks require the involvement of specialized processes, and it provides a clear structure for managing concurrent, complex workflows.

Evaluator-Optimizer

The Evaluator-Optimizer pattern introduces a dual-LLM process where two separate models work in tandem to refine a solution iteratively (see Figure 11-6).

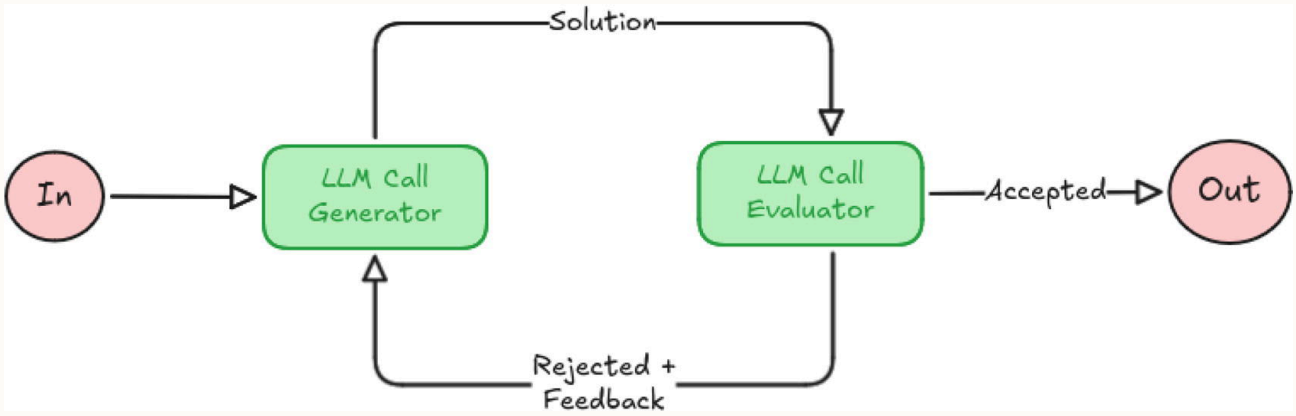


Figure 11-6 Evaluator Optimizer

One model, the Generator LLM, produces initial responses, while the second, the Evaluator LLM, assesses the responses and provides feedback for improvement. This pattern is akin to the refinement process that a human writer goes through when revising and improving their work. The iterative nature of this process makes it highly suitable for tasks that require multiple rounds of critique and enhancement.

When to Use

Here is when to use it:

- **Clear evaluation criteria**: The pattern is most useful when there are specific standards or expectations that the solution must meet.

- **Iterative refinement provides measurable value**: Use it when a task benefits from successive refinements, such as improving accuracy or clarity over multiple iterations.
- **Tasks benefit from multiple rounds of critique**: Use it when a solution requires constant improvement, often through feedback and reworking until the desired outcome is achieved.

This workflow is ideal for tasks such as writing, coding, or designing solutions where refinement is needed to meet high-quality standards.

In the following code, the Evaluator-Optimizer pattern is implemented using Spring AI's ChatClient. The EvaluatorOptimizerWorkflow class simulates an iterative feedback loop where a solution is generated and then evaluated to determine whether it meets the required standards. If the solution is not satisfactory, the process is repeated until the solution passes the evaluation.

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatMode
l) {
    return args -> {
        ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
        System.out.println("Evaluator Optimizer Workflow:");
        evaluatorOptimizer(chatClient);  // Initiates the evaluator-optimizer workflow
    };
}
private void evaluatorOptimizer(ChatClient chatClient) {
    EvaluatorOptimizerWorkflow workflow = new EvaluatorOptimizerWorkflow(chatClie
nt);
    // Start the iterative refinement process with an initial task
    RefinedResponse response = workflow.loop(
            "Create a Java class implementing a thread-safe counter"
    );
    // Output the final solution and the evaluation process
    System.out.println("Final Solution: " + response.solution());
    System.out.println("Evaluation: " + response.chainOfThought());
}
```

EvaluatorOptimizerWorkflow Class

Here is the EvaluatorOptimizerWorkflow class:

```
package com.example.demo.workflow;
import org.springframework.ai.chat.client.ChatClient;
import java.util.Objects;
```

```
public class EvaluatorOptimizerWorkflow {
    private final ChatClient chatClient;
    private final RefinedResponse refinedResponse;
    public EvaluatorOptimizerWorkflow(ChatClient chatClient) {
        this.chatClient = chatClient;
        this.refinedResponse = new RefinedResponse();  // Create a new object to hold the f
inal solution and evaluation
    }
    // Loop for iterative refinement of the solution
    public RefinedResponse loop(String task) {
        String chainOfThought = "NEEDS_IMPROVEMENT";  // Initialize the evaluation as "N
EEDS_IMPROVEMENT"
        String solution = "";
        // Repeat until the solution is considered acceptable ("PASS")
        while (!chainOfThought.equals("PASS")) {
            solution = Objects.requireNonNull(chatClient.prompt(task)
                    .call()
                    .content())  // Generate the initial solution from the task
                .trim();
            refinedResponse.setChainOfThought(solution);  // Set the solution as part of the
chain of thought
            chainOfThought = evaluate(solution, task);  // Evaluate the solution to check if it
needs improvement
        }
        refinedResponse.setSolution(solution);  // Finalize the solution once it's considered
acceptable
        return refinedResponse;
    }
    // Evaluates the generated solution based on the task
    private String evaluate(String solution, String task) {
        return Objects.requireNonNull(chatClient.prompt("Think step by step. Evaluate th
e following solution: "
                + solution + " for task: " + task + ". Reply with PASS if solution is cor
rect, otherwise reply with NEEDS_IMPROVEMENT.")
                .call()
                .content())  // Evaluate the solution and return either "PASS" or "NEEDS_IM
PROVEMENT"
            .trim();
    }
}
```

What's Happening in the Process?

This is what is happening:

1. **CommandLineRunner**: The commandLineRunner method is executed when the application starts. It initiates the **Evaluator-Optimizer** workflow by calling the evaluatorOptimizer() method.
2. **Evaluator-optimizer workflow**: The EvaluatorOptimizerWorkflow class orchestrates the iterative refinement of the solution. It starts by generating a solution (in this case, a Java class for a thread-safe counter) and then checks whether the solution meets the necessary criteria.
3. **Solution generation and evaluation**:
    1. The loop() method continuously refines the solution by calling the **Generator LLM** to generate a solution and then passing it to the **Evaluator LLM** for assessment.
    2. The evaluate() method asks the LLM to analyze the solution and determine whether it meets the required standards. If the solution is deemed inadequate, the process repeats with a new solution. The evaluation result is returned as either "PASS" or "NEEDS_IMPROVEMENT."
4. **Final solution and feedback**: Once the evaluation is “PASS,” the final solution is returned. Along with the solution, the *chain of thought* is recorded, showing how the solution evolved over multiple iterations.
5. **Concurrent feedback loop**: The key feature of this pattern is the feedback loop where the evaluator constantly improves the generator's solution. This iterative refinement process allows for high-quality output, especially when the task benefits from constant critique and improvement.

Key Insights

The following are the key insights:

- **Iterative refinement**: The **Evaluator-Optimizer** pattern excels in scenarios where a task requires constant feedback and iterative improvement. Each solution is refined based on clear evaluation criteria until it meets the desired standard.
- **Dual-layered process**: The pattern uses two LLMs in a complementary way: one for generating potential solutions and another for evaluating and refining those solutions.
- **Clear evaluation**: The process ensures that each step of the task is assessed and improved upon, ensuring the solution evolves over time to meet the highest standards.
- **Use cases**: This pattern is ideal for tasks like writing code, creating documentation, or designing solutions that benefit from step-by-step critique and improvement.

The **Evaluator-Optimizer** pattern is a valuable tool for situations where a solution needs to be progressively refined, ensuring that each iteration improves upon the last until the final result is both accurate and optimal.

Agent Lifecycle in Spring AI

Understanding the lifecycle of an agent is essential for designing systems that are modular, scalable, and responsive. In Spring AI, an agent transitions through several key stages during its operation. Each stage builds on the previous one and contributes to the overall behavior, reliability, and adaptability of the system.

1. **Initialization:** The agent's lifecycle begins with initialization, where the agent is created and configured with the necessary components. This includes selecting the underlying language model, registering available tools (functions or APIs the agent can call), setting up memory storage (such as a vector database), and optionally applying safety guardrails. At this stage, developers define the agent's capabilities and context, ensuring it's properly equipped to process and act on incoming inputs.
2. **Input reception and perception:** Once initialized, the agent listens for and receives input, typically from a user or another system. This input can be a direct prompt, a query, or a command. The agent parses and interprets this input based on its prior knowledge, context, and available tools. If memory is enabled, it can also retrieve relevant past interactions or documents. This perception step sets the stage for informed reasoning and action planning.
3. **Reasoning and planning:** With input in hand, the agent proceeds to reason about the best course of action. This involves selecting appropriate tools, determining the sequence of steps to reach a goal, and making decisions based on the task and context. In more advanced workflows, the agent may dynamically generate plans or select between different workflows or subtasks. This phase leverages the power of the underlying LLM and any structured prompting or instructions provided during initialization.
4. **Action execution:** During this phase, the agent executes its planned actions. This may involve calling tools such as calculators, web search functions, database queries, or even invoking other agents. Each action is performed in response to the reasoning output and can return intermediate results that guide the agent's next step. In Spring AI, this process is often orchestrated through workflows, advisors, or pattern-specific classes.
5. **Observation and feedback:** After executing an action, the agent observes the outcome and incorporates that result into its ongoing reasoning. If the task is not yet complete, the agent may use the new information to adjust its plan, retry a step, or choose a different tool. This feedback loop allows the agent to iterate intelligently,

improving the accuracy and completeness of its response. In evaluation-driven workflows, this is where agents may self-correct or optimize their results.

6. **Response generation:** Once the agent has gathered enough information and completed all necessary steps, it synthesizes the final response. This is typically returned as natural language output, structured data, or a combination of both. The response is informed not only by the original input but also by intermediate tool results, memory references, and any refinements made during the observation-feedback loop.
7. **Memory update (optional):** If the agent is configured with persistent memory, it may store relevant data from the interaction—such as the user's query, contextual embeddings, or results of executed tools. This enables continuity across sessions and allows the agent to personalize future interactions or maintain state over time. Memory updates ensure that the agent becomes more knowledgeable and context-aware with repeated use.
8. **Termination or continuation:** The agent either concludes the interaction or waits for the next input. In stateless applications, this may mark the end of the agent's lifecycle. However, in multistep or session-based systems, the agent may retain context and continue participating in the ongoing dialogue or task. The ability to persist state and remain "alive" between steps is key for building conversational or multi-agent applications.

Spring AI's Implementation Advantages

Spring AI provides a robust framework for implementing advanced AI workflows, offering several key advantages that align with Anthropic's recommendations for effective AI system design. The following are some of the notable benefits.

Model Portability

One of the significant advantages of Spring AI is its *model portability*, which allows developers to easily switch between different AI models through dependencies. This flexibility enables users to integrate multiple LLM providers seamlessly, ensuring that the system remains adaptable to changes in underlying models or requirements.

Here's an example:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
```

```
</dependency>
```

This straightforward dependency configuration ensures that developers can easily swap models and extend their applications without major disruptions.

Structured Output

Spring AI provides *structured output* by leveraging type-safe handling of responses from LLMs. This ensures that responses from the model are well-organized and easy to process, reducing the complexity of managing raw, unstructured data. Type-safe handling ensures that the system can be confident in the integrity of the output, which is particularly important for maintaining reliability across different tasks and workflows.

Here's an example:

```
EvaluationResponse response = chatClient.prompt(prompt)
        .call()
        .entity(EvaluationResponse.class);
```

By using structured output, developers can confidently parse and manipulate responses, improving both development speed and system reliability.

Consistent API

Spring AI ensures that developers have a consistent API across various LLM providers. This uniformity simplifies the integration of different models and tools into an AI system, reducing the learning curve and improving the maintainability of the system. Additionally, Spring AI includes built-in error handling and retry mechanisms, ensuring that the system can recover from transient issues gracefully. Flexible prompt management further enhances the user experience, making it easier to manage and adjust prompts based on dynamic conditions.

Best Practices and Recommendations

Based on Anthropic's research and Spring AI's implementations, the following best practices and recommendations are key to building effective LLM-based systems.

Start Simple

It's essential to start simple when designing AI workflows. Begin with basic, well-defined workflows before introducing complexity. By using the simplest pattern that meets your requirements, you can reduce the chances of errors and ensure the system remains manageable. Complexity should be added only when absolutely necessary, allowing for scalable and maintainable solutions.

Design for Reliability

Reliability is crucial for any enterprise AI application. It is recommended to:

- Implement clear error handling to catch and resolve issues effectively.
- Use type-safe responses wherever possible to avoid runtime errors and ensure that the system can handle various LLM outputs securely.
- Build validation into each step of the workflow to verify that the system is operating as expected at every stage of processing.

Consider Trade-Offs

When building AI systems, there are often trade-offs between various factors:

- **Latency vs. accuracy**: Consider whether the system can tolerate some delay in exchange for higher accuracy or if real-time processing is required.
- **Parallel processing**: Evaluate when parallel processing will enhance performance, especially in cases where tasks can be divided into independent subtasks.
- **Fixed workflows vs. dynamic agents**: Choose between using predefined workflows or more flexible, dynamic agents based on the complexity and requirements of the task at hand.

By following these best practices and leveraging the strengths of Spring AI, developers can build scalable, maintainable, and effective AI systems that meet the needs of modern enterprises.

Advisors API in Spring AI

In Spring AI, advisors are modular components that intercept and optionally transform chat-completion requests and responses, enabling developers to enhance AI applications with reusable, maintainable logic. Functioning in a chain, each advisor can adjust the user's input, format the model's output, share data through a common context, or even block requests when necessary. They are used for tasks such as conversation memory management, retrieval-augmented generation (RAG), and safeguarding against inappro-

priate content. Spring AI supports both nonstreaming advisors, which process complete responses, and streaming advisors, which handle data in chunks for real-time scenarios. Developers can also implement custom advisors to embed domain-specific behavior—such as logging, input rewriting, or reasoning improvements—without altering core application code. Execution order is determined by a configurable priority system, allowing fine-grained control over how transformations are applied across the request-response lifecycle. We will not be going deep with any explanations or code in this section, as exploring and experimenting with advisors is left to the reader and is outside the scope of this chapter.

Conclusion

In conclusion, the use of agents powered by LLMs offers a transformative approach to building AI applications that can adapt, reason, and interact with complex systems. The patterns and workflows outlined in this chapter—such as chain workflows, parallelization, and orchestration—demonstrate how agents can be effectively integrated into enterprise applications using Spring AI. Through the flexible and powerful Advisors API, developers can refine and augment the behaviors of their agents, enhancing their performance and reliability.

Spring AI's design emphasizes simplicity and composability, enabling developers to focus on solving real-world problems without being burdened by overly complex frameworks. By starting with basic workflows and gradually adding sophistication as needed, teams can build scalable and reliable AI systems that deliver measurable value.

As AI continues to evolve, the patterns and techniques explored here lay the foundation for more advanced and intelligent agents. By combining these principles with emerging AI technologies, developers can unlock the full potential of agentic systems, driving innovation and efficiency in AI applications. The journey into the world of intelligent agents is just beginning, and with Spring AI, the possibilities are endless.

Let us now move on to transformers.

Previous chapter

< 10. Swiss-Army LLMs: Tool Calls in Spring AI

Next chapter

12. Quarkus + LangChain4j: Lightning-Fast Gen AI >